Lab 8: Nested Loops and File Handling

Learning Objectives

1. Explain the purpose of file input and output.
2. Distinguish between `ifstream` and `ofstream`.
3. Open, write to, read from, and close text files.
4. Use nested loops to process multiple groups of data.
5. Use accumulators and conditional statements to analyze data.
6. Create programs that save results to files and retrieve them.

Review: Nested Loops From Lecture

- A nested loop is a loop inside another loop.

```
for (int row = 1; row <= 3; row++)
{
    for (int col = 1; col <= 4; col++)
    {
        cout << col << " ";
    }

    cout << endl;
}
```

1 2 3 4
1 2 3 4
1 2 3 4

- The outer loop controls the rows.
- The inner loop controls the columns.
- The inner loop completes all its iterations for each outer-loop iteration.
- Question: How many times does the inner loop execute in total?

3 × 4 = 12 times.

Introduction to File Handling

- So far, we have mainly used:

```
cin >> number;     // Keyboard input
cout << number;    // Screen output
```

- But what if we want to save information so that it can be used later?

- C++ allows programs to read and write files.

Stream	Purpose
`ofstream`	Writes data to a file
`ifstream`	Reads data from a file
`fstream`	Can read and write files

- Required header: `#include <fstream>`

- Variables such as outfile and infile are object names chosen by the programmer.

Writing Data to a File Using `ofstream`

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outfile;

    outfile.open("numbers.txt");

    outfile << 10 << endl;
    outfile << 20 << endl;
    outfile << 30 << endl;

    outfile.close();

    return 0;
}
```

- `open()` opens the file.
- << writes data to the file.
- `close()` closes the file.

Opening an existing file with `ofstream` in this way normally replaces its previous contents.

- The program creates a file named `numbers.txt` containing: 10 20 30

Reading Data from a File Using `ifstream`

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream infile;
    int number;

    infile.open("numbers.txt");

    while (infile >> number)
    {
        cout << number << endl;
    }

    infile.close();

    return 0;
}
```

10
20
30

- How does while (infile >> number) work?
  - It attempts to read the next integer.
  - If reading succeeds, the loop executes.
  - When no more integers can be read, the loop terminates

Question: Why don't we need `number++` inside this loop?

Because the file stream advances as data is read; the loop condition does not depend on incrementing number.

Checking Whether a File Opened Successfully

- A file might fail to open because it does not exist, the path is incorrect, or the program lacks permission.

```
ifstream infile("numbers.txt");

if (!infile)
{
    cout << "Error opening file!" << endl;
    return 1;
}
```

- We should also check output files:

```
ofstream outfile("report.txt");

if (!outfile)
{
    cout << "Error creating file!" << endl;
    return 1;
}
```

Remember:

- `ifstream` does not automatically create a missing input fil
- `ofstream` can create an output file.
- Always check that important file operations succeed.

Live Coding: Weekly Sales Report

Problem

Write a C++ program that:

1. Asks how many stores will be processed.
2. Asks how many days of sales will be recorded for each store.
3. Uses nested loops to collect daily sales.
4. Calculates total sales for each store.
5. Uses `if/else` to determine performance:
    - Total sales of $500 or more: Good
    - Otherwise: Needs Improvement
6. Writes each store's total and performance to `sales_report.txt`.
7. Reads the saved report and displays it on the screen.

Think Before You Code

- Which loop processes stores?
- Which loop processes days?
- Where should total be initialized?
- Which stream writes to the file?
- Which stream reads from the file?

Outer for loop processes stores.

Inner for loop processes days.

`total = 0;` inside the outer loop, before the inner loop.

`ofstream` writes to the file.

`ifstream` reads from the file.

Part 1: Getting input and opening the output file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
    int numStores, numDays;
    double sales, total;

    cout << "Enter number of stores: ";
    cin >> numStores;

    cout << "Enter number of days: ";
    cin >> numDays;

    if (numStores <= 0 || numDays <= 0)
    {
        cout << "Invalid input!" << endl;
        return 1;
    }

    ofstream outfile("sales_report.txt");

    if (!outfile)
    {
        cout << "Error creating file!" << endl;
        return 1;
    }
```

Part 2: Nested loops, accumulator, and conditional statements

```
for (int store = 1; store <= numStores; store++)
   {
       total = 0;

       for (int day = 1; day <= numDays; day++)
       {
           cout << "Store " << store
                << ", Day " << day
                << " sales: ";

           cin >> sales;

           total += sales;
       }
       outfile << "Store " << store
               << ": Total = $" << total;

       if (total >= 500)
       {
           outfile << " - Good" << endl;
       }
       else
       {
           outfile << " - Needs Improvement" << endl;
       }
   }
   outfile.close();
```

- Why is total = 0; inside the outer loop?

To reset the total for each new store.

- What happens if we move it outside the outer loop?

Sales from previous stores will accumulate, giving incorrect totals.

- Why is the if/else after the inner loop?

To classify each store after all its daily sales have been added.

Part 3: Reading and displaying the saved report

```
    ifstream infile("sales_report.txt");

    if (!infile)
    {
        cout << "Error opening report!" << endl;
        return 1;
    }

    string line;

    cout << "\nWeekly Sales Report\n";

    while (getline(infile, line))
    {
        cout << line << endl;
    }

    infile.close();

    return 0;
}
```

Example input:
Enter number of stores: 2
Enter number of days: 2
Store 1, Day 1 sales: 200
Store 1, Day 2 sales: 350
Store 2, Day 1 sales: 100
Store 2, Day 2 sales: 150

File contents and displayed report:

Store 1: Total = $550 - Good
Store 2: Total = $250 - Needs Improvement

- Question: Why use `getline(infile, line)` rather than `infile >> line`?

  Because `getline()` reads the entire line, including spaces between word

Complete Weekly Sales Report

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
    int numStores, numDays;
    double sales, total;

    cout << "Enter number of stores: ";
    cin >> numStores;

    cout << "Enter number of days: ";
    cin >> numDays;

    if (numStores <= 0 || numDays <= 0)
    {
        cout << "Invalid input!" << endl;
        return 1;
    }

    // Open file for writing
    ofstream outfile("sales_report.txt");

    if (!outfile)
    {
        cout << "Error creating file!" << endl;
        return 1;
    }

    // Process each store
    for (int store = 1; store <= numStores; store++)
    {
        total = 0;

        // Collect daily sales
        for (int day = 1; day <= numDays; day++)
        {
            cout << "Store " << store
                 << ", Day " << day << " sales: ";
            cin >> sales;
            total += sales;
        }

        // Write results to file
        outfile << "Store " << store
                << ": Total = $" << total;

        if (total >= 500)
        {
            outfile << " - Good" << endl;
        }
        else
        {
            outfile << " - Needs Improvement" << endl;
        }
    }
    outfile.close();

    // Open file for reading
    ifstream infile("sales_report.txt");

    if (!infile)
    {
        cout << "Error opening report!" << endl;
        return 1;
    }

    string line;

    cout << "\nWeekly Sales Report\n";

    while (getline(infile, line))
    {
        cout << line << endl;
    }

    infile.close();
    return 0;
}
```

Input

Enter number of stores: 2
Enter number of days: 2
Store 1, Day 1 sales: 200
Store 1, Day 2 sales: 350
Store 2, Day 1 sales: 100
Store 2, Day 2 sales: 150

Output

Weekly Sales Report
Store 1: Total = $550 - Good
Store 2: Total = $250 - Needs Improvement

In-Class Exercise 1: Employee Work Hours Report

Write a C++ program that:

1. Asks the user for the number of employees.
2. Asks for the number of workdays.
3. Uses nested loops to collect each employee's daily work hours.
4. Calculates total hours worked by each employee.
5. Uses if/else to classify:
    - More than 40 hours: Overtime
    - Otherwise: Regular
6. Writes each employee's total hours and classification to employee_report.txt.

Example:

```
Enter number of employees: 2
Enter number of days: 3

Employee 1, Day 1 hours: 8
Employee 1, Day 2 hours: 9
Employee 1, Day 3 hours: 8

Employee 2, Day 1 hours: 15
Employee 2, Day 2 hours: 14
Employee 2, Day 3 hours: 13
```

Expected file contents:

```
Employee 1: 25 hours - Regular
Employee 2: 42 hours - Overtime
```

Hint: Follow the structure of the Weekly Sales Report, but change the input, calculations, and classification.

In-Class Exercise 2: Electricity Consumption Report

Write a C++ program that:

1. Asks how many households will be processed.
2. Asks how many days of electricity usage will be recorded.
3. Uses nested loops to collect daily electricity consumption in kWh.
4. Calculates total consumption for each household.
5. Uses `if/else` to classify:
   - Total greater than 100 kWh: High Usage
   - Otherwise: Normal Usage
6. Writes the results to `electricity_report.txt`.
7. Uses ifstream and `getline()` to read the file and display its contents.

Example input:

```
Enter number of households: 2
Enter number of days: 2

Household 1, Day 1 usage: 30
Household 1, Day 2 usage: 40

Household 2, Day 1 usage: 60
Household 2, Day 2 usage: 55
```

Expected report:

```
Household 1: 70 kWh - Normal Usage
Household 2: 115 kWh - High Usage
```

Close the output file before reopening it with `ifstream`.

Optional Practice: Student Attendance Report

Write a C++ program that:

1. Asks for the number of students and the number of class meetings.
2. Uses nested loops to record attendance for each student.
3. For each class meeting, enter:
    - 1 for Present
    - 0 for Absent
4. Counts how many classes each student attended.
5. Uses if/else to classify:
    - Attended all classes: Perfect Attendance
    - Otherwise: Has Absences
6. Saves the results to attendance.txt.
7. Reads and displays the saved report.

Example for three class meetings:

```
Student 1: 3/3 - Perfect Attendance
Student 2: 2/3 - Has Absences
```

Challenge: Can you use a while loop to reject attendance values other than 0 and 1?

Summary

Today we practiced:

- `ofstream` for writing files
- `ifstream` for reading files
- `open()` and `close()`
- `while (infile >> value)`
- `while (getline(infile, line))`
- `Nested loops` and `accumulators`
- `if/else` statements for classifying results